CLOUD-BASED FILE STORAGE MANAGEMENT SYSTEM

AATMini ProjectReport

Technology: MySQL

Title: Cloud-Based File Storage Management System

Student Name	MUTHU KUMARAN
Reg. No.	711525BCS099
Department / Class	BE. CSE / C
Subject Code	B25CST302
Subject	Database Management System
College	Kalaignarkarunanidhi Institute of Technology

Academic mini project demonstrating relational database design and SQL-based file-storage reports.

1. Abstract

TheCloud-Based File Storage Management System is a database project designed to organize file metadata, user accounts, folders, file versions, share links, and activity records. The database stores information about uploaded files, including file name, type, size, upload date, and storage location. SQL queries generate useful reports such as total files, storage used by each user, file-type distribution, recent uploads, shared files, and activity summaries. This academic model represents the management layer of a cloud storage service; the actual file bytes would be stored in cloud object storage or afile server.

2. Objectives

• Maintain user, folder, file, version, sharing, and activity records.

• Organize files by user and folder.

• Track file size, type, upload date, and storage location.

• Maintain file-version history for updated files.

• Generate storage-usage and file-type reports.

• Trackshare-linkstatusandfile-related activity.

3. Software Requirements

Component	Requirement
Database	MySQL 8.x or compatible version
Interface	MySQL Workbench, phpMyAdmin, or MySQL command line
Operating System	Windows, Linux, or macOS
Input	SQL statements and sample records
Output	Database tables and management reports

4. Database Design

ThedatabaseisnamedCloudStorageDB. It contains six related tables.

Table	Purpose	Primary key
Users	Account details and storage quota	user_id
Folders	Folder names and parent-folder links	folder_id
Files	File metadata and current status	file_id
FileVersions	Version number, size, and upload date	version_id
ShareLinks	Share token, expiry, and active status	share_id
ActivityLogs	User actions and timestamps	activity_id

4.1 Relationship Summary

• Oneuser can ownmanyfolders andfiles.

• Folders can be nested using a parent_folder_id reference.

• One file can have multiple versions.

• One file can have multiple share links.

• Activity log entries canbe associatedwithauserand, optionally, a file.

5. Complete SQL Program

Run the database andtable-creation statementsfirst, followed by the sample data. The sample records are fictional and are used only to demonstrate the project.

5.1 Create Database and Tables

```
CREATE DATABASE CloudStorageDB;
USE CloudStorageDB;

CREATE TABLE Users (
  user_id INT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) UNIQUE NOT NULL,
  storage_quota_mb INT NOT NULL
);

CREATE TABLE Folders (
  folder_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  folder_name VARCHAR(100) NOT NULL,
  parent_folder_id INT NULL,
  created_at DATE,
  FOREIGN KEY (user_id) REFERENCES Users(user_id),
  FOREIGN KEY (parent_folder_id) REFERENCES Folders(folder_id)
);

CREATE TABLE Files (
  file_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  folder_id INT NULL,
  file_name VARCHAR(150) NOT NULL,
  file_type VARCHAR(20),
  size_mb DECIMAL(10,2) NOT NULL,
  storage_path VARCHAR(255) NOT NULL,
  uploaded_at DATE,
  file_status VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (user_id) REFERENCES Users(user_id),
  FOREIGN KEY (folder_id) REFERENCES Folders(folder_id)
);
```

5.2 Create Remaining Tables

```
CREATETABLE FileVersions (
  version_id INT PRIMARY KEY,
  file_id INT NOT NULL,
  version_number INT NOT NULL,
  size_mb DECIMAL(10,2) NOT NULL,
  uploaded_at DATE,
  UNIQUE (file_id, version_number),
  FOREIGN KEY (file_id) REFERENCES Files(file_id)
);

CREATE TABLE ShareLinks (
  share_id INT PRIMARY KEY,
  file_id INT NOT NULL,
  share_token VARCHAR(80) UNIQUE NOT NULL,
  expires_on DATE,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (file_id) REFERENCES Files(file_id)
);

CREATE TABLE ActivityLogs (
  activity_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  file_id INT NULL,
  action_name VARCHAR(40) NOT NULL,
  action_time DATETIME NOT NULL,
  FOREIGN KEY (user_id) REFERENCES Users(user_id),
  FOREIGN KEY (file_id) REFERENCES Files(file_id)
);
```

5.3 Insert Sample Data

```
INSERTINTO Users VALUES
(1, 'Arun Kumar', 'arun@example.com', 5000),
(2, 'Bala Devi', 'bala@example.com', 3000),
(3, 'Kavin Raj', 'kavin@example.com', 2000);

INSERT INTO Folders VALUES
(10, 1, 'Documents', NULL, '2026-10-01'),
(11, 1, 'College', 10, '2026-10-02'),
(20, 2, 'Photos', NULL, '2026-10-03');

INSERT INTO Files VALUES
(101, 1, 10, 'project_report.pdf', 'PDF', 12.50,
'/cloud/user1/project_report.pdf', '2026-10-05', 'Active'),
(102, 1, 11, 'dbms_notes.docx', 'DOCX', 4.20,
'/cloud/user1/college/dbms_notes.docx', '2026-10-08', 'Active'),
(103, 2, 20, 'vacation.jpg', 'JPG', 8.75,
'/cloud/user2/photos/vacation.jpg', '2026-10-10', 'Active'),
(104, 2, NULL, 'backup.zip', 'ZIP', 120.00,
'/cloud/user2/backup.zip', '2026-10-12', 'Active'),
(105, 3, NULL, 'old_notes.pdf', 'PDF', 6.00,
'/cloud/user3/old_notes.pdf', '2026-10-14', 'Deleted');

INSERT  INTO  FileVersions  VALUES
(1, 101, 1, 10.00, '2026-10-04'),
(2, 101, 2, 12.50, '2026-10-05'),
(3,  102,  1,  4.20,  '2026-10-08'),
(4, 103, 1, 8.75, '2026-10-10');

INSERT INTO ShareLinks VALUES
(1,  101,  'SHARE-ABC101',  '2026-12-31',  TRUE),
(2,  103,  'SHARE-XYZ103',  '2026-11-30',  TRUE),
(3, 104, 'SHARE-OLD104', '2026-09-30', FALSE);

INSERT INTO ActivityLogs VALUES
(1,  1,  101,  'UPLOAD', '2026-10-05 09:15:00'),
(2, 1, 102, 'UPLOAD', '2026-10-08 10:20:00'),
(3, 2, 103, 'SHARE',   '2026-10-10 14:00:00'),
 (4, 2, 104, 'UPLOAD', '2026-10-12 16:45:00'),
 (5, 3, 105, 'DELETE', '2026-10-15 11:30:00');
```

6. Basic SQL Queries and Expected Output

6.1 Display all active files

```
SELECT file_id, file_name, file_type, size_mb, uploaded_at
FROM Files
WHERE file_status = 'Active'
ORDER BY file_id;
```

file_id	file_name		size_mb	uploaded_at
1	project_report.pdf		12.50	2026-10-05
2	dbms_notes.docx		4.20	2026-10-08
3	vacation.jpg		8.75	2026-10-10
4	backup.zip		120.00	2026-10-12

6.2 Display users and their storage quota

```
SELECT user_id, full_name, email,storage_quota_mb
FROM Users
ORDER BY user_id;
```

user_id	full_name	email	storage_quota_mb
1	Arun Kumar	arun@example.com	5000
2	Bala Devi	bala@example.com	3000
3	Kavin Raj	kavin@example.com	2000

6.3 Display files with owner names

```
SELECT u.full_name, f.file_name,f.file_type, f.size_mb
FROM Users u
JOIN Files f ON u.user_id = f.user_id
ORDER BY u.user_id, f.file_id;
```

full_name	file_name	file_type	size_mb
Arun Kumar	project_report.pdf	PDF	12.50
Arun Kumar	dbms_notes.docx	DOCX	4.20
Bala Devi	vacation.jpg	JPG	8.75
Bala Devi	backup.zip	ZIP	120.00
Kavin Raj	old_notes.pdf	PDF	6.00

7. Analytics Queries and Expected Output

7.1 Total active storage used (MB)

```
SELECT SUM(size_mb) AS total_storage_mb
FROM Files
WHERE file_status = 'Active';
```

Expected output: total_storage_mb = 145.45

This query adds the sizes of all active files and excludes the file marked Deleted.

7.2 Storage used by each user

```
SELECT u.full_name, COALESCE(SUM(f.size_mb), 0) AS used_mb
FROM Users u
LEFT JOIN Files f
  ON u.user_id = f.user_id AND f.file_status = 'Active'
GROUP BY u.user_id, u.full_name
ORDER BY u.user_id;
```

full_name	used_mb
Arun Kumar	16.70
Bala Devi	128.75
Kavin Raj	0.00

7.3 File count by file type

```
SELECT file_type, COUNT(*) ASfile_count
FROM Files
WHERE file_status = 'Active'
GROUP BY file_type
ORDER BY file_type;
```

file_type	file_count
DOCX	1
JPG PDF	1
ZIP	1
	1

7.4 Find files larger than 10 MB

```
SELECT file_name, size_mb
FROM Files
WHERE file_status = 'Active' AND size_mb > 10
ORDER BY size_mb DESC;
```

file_name	size_mb
backup.zip	120.00
project_report.pdf	12.50

7.5 List active share links

```
SELECT f.file_name, s.share_token, s.expires_on
FROM ShareLinks s
JOIN Files f ON s.file_id = f.file_id
WHERE s.is_active = TRUE
ORDER BY s.share_id;
```

file_name	share_token	expires_on
project_report.pdf	SHARE-ABC101	2026-12-31
vacation.jpg	SHARE-XYZ103	2026-11-30

7.6 Show recent uploads

```
SELECT file_name, uploaded_at,size_mb
FROM Files
WHERE file_status = 'Active'
ORDER BY uploaded_at DESC
LIMIT 3;
```

file_name	uploaded_at	size_mb
backup.zip	2026-10-12	120.00
vacation.jpg	2026-10-10	8.75
dbms_notes.docx	2026-10-08	4.20

7.7 Count activity by action

```
SELECT action_name, COUNT(*)AStotal_actions
FROM ActivityLogs
GROUP BY action_name
ORDER BY action_name;
```

action_name	total_actions
DELETE	1
SHARE	1
UPLOAD	3

7.8 Display version history for a file

```
SELECT f.file_name, v.version_number,v.size_mb, v.uploaded_at
FROM Files f
JOIN FileVersions v ON f.file_id = v.file_id
WHERE f.file_id = 101
ORDER BY v.version_number;
```

file_name	version_number	size_mb	uploaded_at
project_report.pdf	1	10.00	2026-10-04
project_report.pdf	2	12.50	2026-10-05

7.9 Display files and folder names

```
SELECT f.file_name, COALESCE(fd.folder_name, 'Root') AS folder_name
FROM Files f
LEFT JOIN Folders fd ON f.folder_id = fd.folder_id
WHERE f.file_status = 'Active'
ORDER BY f.file_id;
```

file_name	folder_name
project_report.pdf	Documents
dbms_notes.docx	College
vacation.jpg	Photos
backup.zip	Root

7.10 Identify files without an active share link

```
SELECT f.file_name
FROM Files f
LEFT JOIN ShareLinks s
  ON f.file_id = s.file_id AND s.is_active = TRUE
WHERE f.file_status = 'Active' AND s.share_id IS NULL
ORDER BY f.file_id;
```

file_name
dbms_notes.docx
backup.zip

8. Optional Improvement: Soft Delete a File

Insteadofphysicallydeleting afilerecord,the system canmarkit asDeleted. This keeps a record for auditing. In a real application, check user permissions and update the metadata and cloud object storage consistently.

```
START TRANSACTION;

UPDATE Files
SET file_status = 'Deleted'
WHERE file_id = 102 AND user_id = 1;

SELECT file_name, file_status
FROM Files
WHERE file_id = 102;

COMMIT;
```

Expected result after the update: dbms_notes.docx has file_status = Deleted. Run the update only when you intend to change the sample record.

9. How to Run the Project

• Install MySQL Serverand openMySQLWorkbench or another MySQL client.

• Create a new SQL query tab.

• Run the CREATE DATABASE and CREATE TABLE statements in order.

• Run all INSERT statements to load the sample records.

• Execute the basic queries to inspect users, folders, and files.

• Run the analytics queries and compare the results with the expected outputs.

• For a freshtest, useanew database name or remove the old sample database only if its data is no longer needed.

10. Advantages

• Stores filemetadata inorganized relational tables.

• Uses primary and foreign keys to maintain relationships.

• Supports folder organization and file-version tracking.

• Provides reports for storage usage and file types.

• Tracks share links and user activities.

• Canbe extended with a web ormobileinterfaceandcloudobjectstorage.

11. Limitations and Future Enhancements

This academic version storesfilemetadata andsamplestorage pathsinMySQL; it does not upload or download real files. It does not include authentication, encryption, access-control enforcement, billing, malware scanning, or integration with a live cloud provider. Future improvements can add secure login, role-based access, password reset, encryption at rest and in transit, expiring share links, audit dashboards, automated quota checks, backups, and integration with services such as Amazon S3, Azure Blob Storage, or Google Cloud Storage.

12. Conclusion

TheCloud-Based File Storage Management System demonstrates how a relational database can organize the metadata needed by a cloud file-storage service. The Users, Folders, Files, FileVersions, ShareLinks, and ActivityLogs tables represent ownership, folder structure, file details, version history, sharing, and activity. The SQL queries report active storage usage, storage by user, file-type counts, large files, recent uploads, and share-link status. This project is suitable as a beginner-level DBMS mini project and can be expanded into a complete cloud-enabled application.

Project Summary

Item	Description
Project title	Cloud-Based File Storage Management System
Database	MySQL / CloudStorageDB
Main concepts	Relational tables, primary keys, foreign keys, joins, grouping, aggregate functions
Main reports	Storage usage, file types, recent uploads, share links, version history
Future scope	Secure web/mobile interface and cloud object-storage integration

— End of Report —